

 Resumen General Indispensable — Sistema Canje FTT

Proyecto: Sistema de Gestión de Canje de Entradas (Canje FTT) **Evento:** Concierto The Lumineers **Pasante:** Gabriel **Tutor Empresarial:** Miguel Vivanco **Período de pasantía:** 01/06/2026 — 10/07/2026 (6 semanas, 180 horas)

1. ¿Qué es el sistema? (en una frase)

Aplicación web que permite al personal de puntos de venta **buscar, validar, canjear e imprimir** las entradas compradas en línea para un concierto, con **control de usuarios por roles, auditoría completa de cada operación y estadísticas en tiempo real** para la administración.

2. Problema que resuelve

Antes del sistema, el canje de entradas se controlaba manualmente (listas en papel / hojas de cálculo), lo que producía:

- Riesgo de **dobles canje** de una misma entrada.
- Sin trazabilidad: no se sabía **quién entregó qué entrada, cuándo ni a quién**.
- Búsquedas lentas del comprador entre miles de registros.
- Sin estadísticas para la administración durante el operativo.

El sistema elimina estos problemas: cada entrada tiene un estado único (**canjeado: true/false**), cada canje queda registrado en auditoría con usuario, fecha, IP y datos de quien retira, y el doble canje es **imposible a nivel de aplicación**.

3. Requisitos fundamentales

Requisitos funcionales (los indispensables)

#	Requisito	Módulo
RF1	Autenticación con usuario/contraseña y sesión JWT	Auth
RF2	Control de acceso por roles: Jefe (admin total) y Staff (solo canje/consulta)	Auth
RF3	Importación masiva de entradas desde CSV oficial de la ticketera	Scripts/Setup
RF4	Búsqueda de entradas por Ticket ID, cédula, email, nombre y localidad	Tickets
RF5	Canje individual: valida que la entrada exista y no esté canjeada, registra quién retira (Titular / Titular Compra / Otro + parentesco) y celular de contacto	Tickets
RF6	Canje masivo (varias entradas de una misma transacción en una sola operación)	Tickets
RF7	Impresión y reimpresión de entradas (reimpresión solo con permiso y motivo)	Tickets

#	Requisito	Módulo
RF8	Auditoría: registro inmutable de canjes, logins, creación de usuarios, cambios de contraseña	Audit
RF9	Gestión de usuarios (crear, editar, desactivar) — solo Jefe	Users
RF10	Dashboard con estadísticas: canjeados vs. pendientes, por localidad, por punto de venta	Dashboard
RF11	Actualización en tiempo real entre operadores (Socket.IO): si un punto canjea, los demás lo ven al instante	Tiempo real

Requisitos no funcionales (los indispensables)

- **Seguridad:** contraseñas hasheadas (bcryptjs), tokens JWT con expiración, Helmet, rate limiting, CORS configurado, validación de entrada en frontend y backend.
- **Integridad:** una entrada canjeada no puede volver a canjearse (verificación en servidor, no solo en pantalla).
- **Rendimiento:** búsquedas con índices en MongoDB y paginación; canje masivo con **bulkWrite** (una sola operación a la BD).
- **Trazabilidad:** todo cambio de estado queda en **AuditLog** con usuario, IP y timestamp.
- **Usabilidad:** interfaz simple para operadores bajo presión (evento en vivo), alertas claras (SweetAlert2).

4. Tecnologías (stack MERN)

Backend

Tecnología	Uso
Node.js + Express 4	Servidor y API REST
MongoDB Atlas + Mongoose	Base de datos en la nube y ODM
JWT + bcryptjs	Autenticación y hash de contraseñas
Socket.IO	Actualizaciones en tiempo real
Helmet, CORS, express-rate-limit, express-validator	Seguridad
Winston	Logging estructurado
csv-parser + Multer	Importación del CSV de entradas

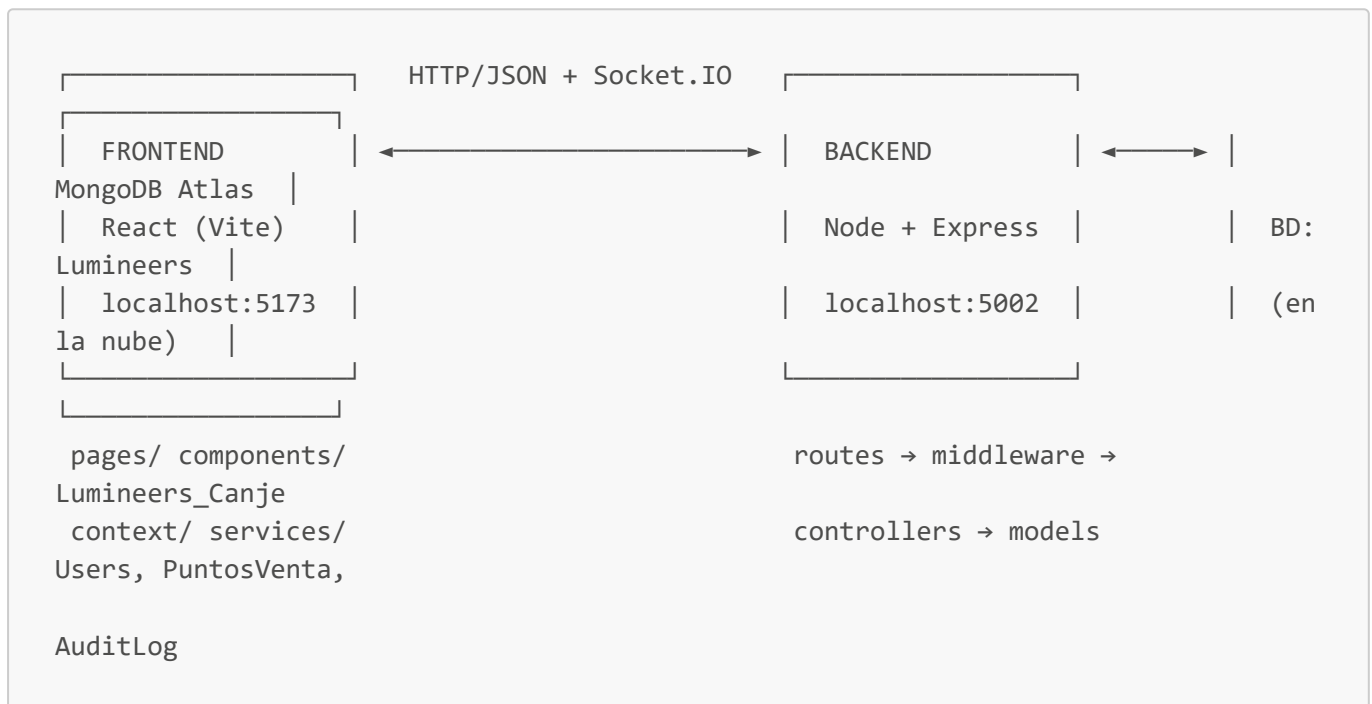
Frontend

Tecnología	Uso
React 18 + Vite	Interfaz de usuario (SPA)
React Router v6	Navegación y rutas protegidas
Bootstrap 5 + React Bootstrap	Diseño responsivo

Tecnología	Uso
Context API + React Query	Estado global y caché de datos
Chart.js	Gráficos del dashboard
Axios	Consumo de la API
Socket.IO client	Tiempo real

5. Arquitectura

Arquitectura **cliente-servidor en 3 capas**, con separación total entre frontend y backend (comunicación solo por API REST JSON + WebSockets):



Patrón del backend (MVC): **routes/** define los endpoints → **middleware/** valida token y rol → **controllers/** ejecuta la lógica de negocio → **models/** (esquemas Mongoose) persiste en MongoDB.

Patrón del frontend: **pages/** (una por pantalla) → **services/** (llamadas Axios a la API) → **context/** (sesión del usuario) → **components/** (Layout, rutas protegidas, tablas reutilizables).

6. Base de datos (MongoDB — BD Lumineers)

Colección	Contenido clave
Lumineers_Canje	Entradas del CSV: nombre, email, cédula, Ticket ID (único), Seat (localidad), Transaction ID , y estado de canje (canjeado , fechaCanje , usuarioCanje , quienRetira , celular)
Users	Usuarios: nombre, usuario, password (hash), rol (jefe/staff), activo, primer acceso
PuntosVenta	Puntos de venta con sus localidades (extraídas automáticamente del CSV)

Colección	Contenido clave
AuditLog	Cada acción: tipo, usuario, ticketId, detalles, IP, timestamp

Decisión de diseño importante: las **localidades no están hardcodedas** — se extraen dinámicamente de la columna **Seat** del CSV, por lo que el sistema sirve para cualquier evento futuro con solo cambiar el CSV.

7. Flujo principal (canje de entrada)

1. Operador (Staff) inicia sesión → JWT.
2. Busca la entrada por cédula, Ticket ID o email.
3. Verifica la identidad del cliente contra los datos en pantalla.
4. Presiona **Canjear** → llena formulario obligatorio: quién retira (Titular / Titular Compra / Otro + parentesco) y celular.
5. El servidor valida que la entrada **no esté ya canjeada**; si lo está, rechaza con error.
6. Se marca **canjeado = true**, se registra usuario/fecha/punto, se crea el registro de **auditoría** y se emite el evento en **tiempo real** a los demás puntos.
7. Se imprime la entrada física.

8. Cómo ejecutar el sistema (demo)

```
# Terminal 1 – Backend (puerto 5002)
cd backend
npm run dev

# Terminal 2 – Frontend (puerto 5173)
cd frontend
npm run dev
```

Abrir **http://localhost:5173** en el navegador.

- Si la BD está vacía, primero importar el CSV: `cd backend && node src/scripts/setup.js ../../LUMINEERS.csv`
- Requiere conexión a internet (la BD es MongoDB Atlas, en la nube).

9. Resultados del proyecto

- Sistema **funcional y desplegable** (configuración lista para Render/Netlify).
- ~4.000+ entradas reales importadas desde el CSV oficial.
- Doble canje imposible a nivel de servidor.
- Trazabilidad completa: cada entrega tiene responsable, fecha, IP y datos de quien retiró.
- Administración con visibilidad en tiempo real del avance del canje por localidad.
- Reutilizable para futuros eventos cambiando únicamente el archivo CSV.